



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**HireCode- *SIMULADOR DE ENTREVISTAS
TÉCNICAS***

Curso: *PATRONES DE SOFTWARE Sec. A*

Docente: *PATRICK JOSE CUADROS QUIROGA*

Integrantes:

Diego Fernando Castillo Mamani (2022073895)

Sergio Alberto Colque Ponce (2022073503)

Renzo Antonio Antayhua Mamani (2022073504)

**Tacna – Perú
2026**

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MPV	ELV	ARV	10/10/2020	Versión Original

Sistema HireCode-*SIMULADOR DE ENTREVISTAS TECNICAS*

Documento de Especificación de Requerimientos de Software

Versión {1.0}

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MPV	ELV	ARV	10/10/2020	Versión Original

INDICE GENERAL

INTRODUCCION	4
I. Generalidades de la Empresa	5
1. Nombre de la Empresa	5
2. Vision	5
3. Mision	5
4. Organigrama	5
II. Visionamiento de la Empresa	5
1. Descripcion del Problema	5
2. Objetivos de Negocios	5
3. Objetivos de Diseño	5
4. Alcance del proyecto	5
5. Viabilidad del Sistema	5
6. Informacion obtenida del Levantamiento de Informacion	6
III. Análisis de Procesos	6
a) Diagrama del Proceso Actual – Diagrama de actividades	6
b) Diagrama del Proceso Propuesto – Diagrama de actividades Inicial	7
IV Especificacion de Requerimientos de Software	7
a) Cuadro de Requerimientos funcionales Inicial	7
b) Cuadro de Requerimientos No funcionales	7
c) Cuadro de Requerimientos funcionales Final	8
d) Reglas de Negocio	9
V Fase de Desarrollo	12
	3

1. Perfiles de Usuario	12
2. Modelo Conceptual	5
a) Diagrama de Paquetes	5
b) Diagrama de Casos de Uso	12
c) Escenarios de Caso de Uso (narrativa)	14
3. Modelo Logico	23
a) Analisis de Objetos	23
b) Diagrama de Actividades con objetos	32
c) Diagrama de Secuencia	37
d) Diagrama de Clases	42
CONCLUSIONES	46
RECOMENDACIONES	46
BIBLIOGRAFIA	46
WEBGRAFIA	46

INTRODUCCIÓN

El presente informe técnico tiene como objetivo documentar el análisis, diseño e implementación del proyecto denominado Simulador de Entrevistas Técnicas. Esta plataforma web interactiva ha sido desarrollada para apoyar tanto a reclutadores de tecnología como a desarrolladores de software en la preparación y ejecución de evaluaciones técnicas de programación de manera eficiente y estructurada.

A lo largo del documento se describen las generalidades del sistema, el proceso de visionamiento, los requerimientos funcionales y no funcionales, así como los diagramas de análisis y diseño que sustentan la arquitectura de la solución propuesta.

I. GENERALIDADES DE LA EMPRESA

1. Nombre de la Empresa

TechSim Solutions

2. Visión

Ser la plataforma líder en simulación de entrevistas técnicas de programación en el mercado latinoamericano, reconocida por ofrecer un entorno de evaluación justo, automatizado y accesible que impulse el talento tecnológico de la región.

3. Misión

Desarrollar y mantener una plataforma web interactiva que permita a reclutadores y desarrolladores de software gestionar, ejecutar y evaluar retos técnicos de programación en tiempo real, mediante tecnologías modernas de ejecución de código y autenticación segura, contribuyendo a procesos de selección más objetivos y eficientes.

4. Organigrama

El proyecto está organizado por los siguientes roles:

- Líder de Proyecto — Coordinación general, arquitectura del sistema y toma de decisiones técnicas.
- Desarrollador Backend — Implementación de la lógica del servidor con Node.js, Express y MySQL.
- Desarrollador Frontend — Diseño de vistas EJS, estilos CSS y experiencia de usuario.
- Especialista en Integración — Conexión con APIs externas: Judge0 (ejecución de código) y Google Identity Services (autenticación OAuth2).

II. VISIONAMIENTO DE LA EMPRESA

1. Descripción del Problema

Actualmente, los procesos de evaluación técnica en el área de desarrollo de software presentan diversas deficiencias. Los reclutadores no cuentan con herramientas centralizadas para organizar, reutilizar y ejecutar ejercicios algorítmicos en tiempo real. Por su parte, los candidatos enfrentan entornos poco realistas que no reflejan las condiciones de una entrevista técnica profesional.

Esta problemática genera pérdidas de tiempo, falta de estandarización en las evaluaciones y resultados poco confiables para ambas partes involucradas.

2. Objetivos de Negocios

- Reducir el tiempo invertido en la preparación y ejecución de entrevistas técnicas.
- Centralizar la gestión de bancos de preguntas y ejercicios algorítmicos.
- Garantizar evaluaciones objetivas y reproducibles mediante ejecución automática de código.

3. Objetivos de Diseño

- Implementar una interfaz intuitiva con tema oscuro (Dark/Glassmorphism) orientada a programadores.
- Diseñar una arquitectura MVC escalable usando Node.js, Express y EJS.
- Integrar la API de Judge0 para la ejecución remota de código en múltiples lenguajes.

4. Alcance del Proyecto

El sistema contempla la gestión de usuarios autenticados mediante Google OAuth2, la creación y administración de bancos de preguntas (denominados Bancos), la resolución de retos algorítmicos en un editor en línea con soporte para JavaScript, Python, Java, C++ y C#, y la evaluación automática de las soluciones mediante entrada/salida estándar (STDIN/STDOUT).

Queda fuera del alcance la implementación de videoconferencia, análisis de voz, o integración con plataformas de reclutamiento de terceros en esta versión.

5. Viabilidad del Sistema

Técnicamente, el proyecto es viable dado que todas las tecnologías empleadas (Node.js, MySQL, Judge0 API, Google Identity Services) son de acceso libre o cuentan con planes gratuitos suficientes para el prototipo. Operativamente, el equipo de desarrollo posee los conocimientos necesarios para implementar y mantener el sistema dentro de los plazos académicos establecidos.

6. Información Obtenida del Levantamiento de Información

Para identificar los requerimientos del sistema, se realizaron las siguientes actividades de levantamiento de información:

Entrevistas Se entrevistó a 3 perfiles clave:

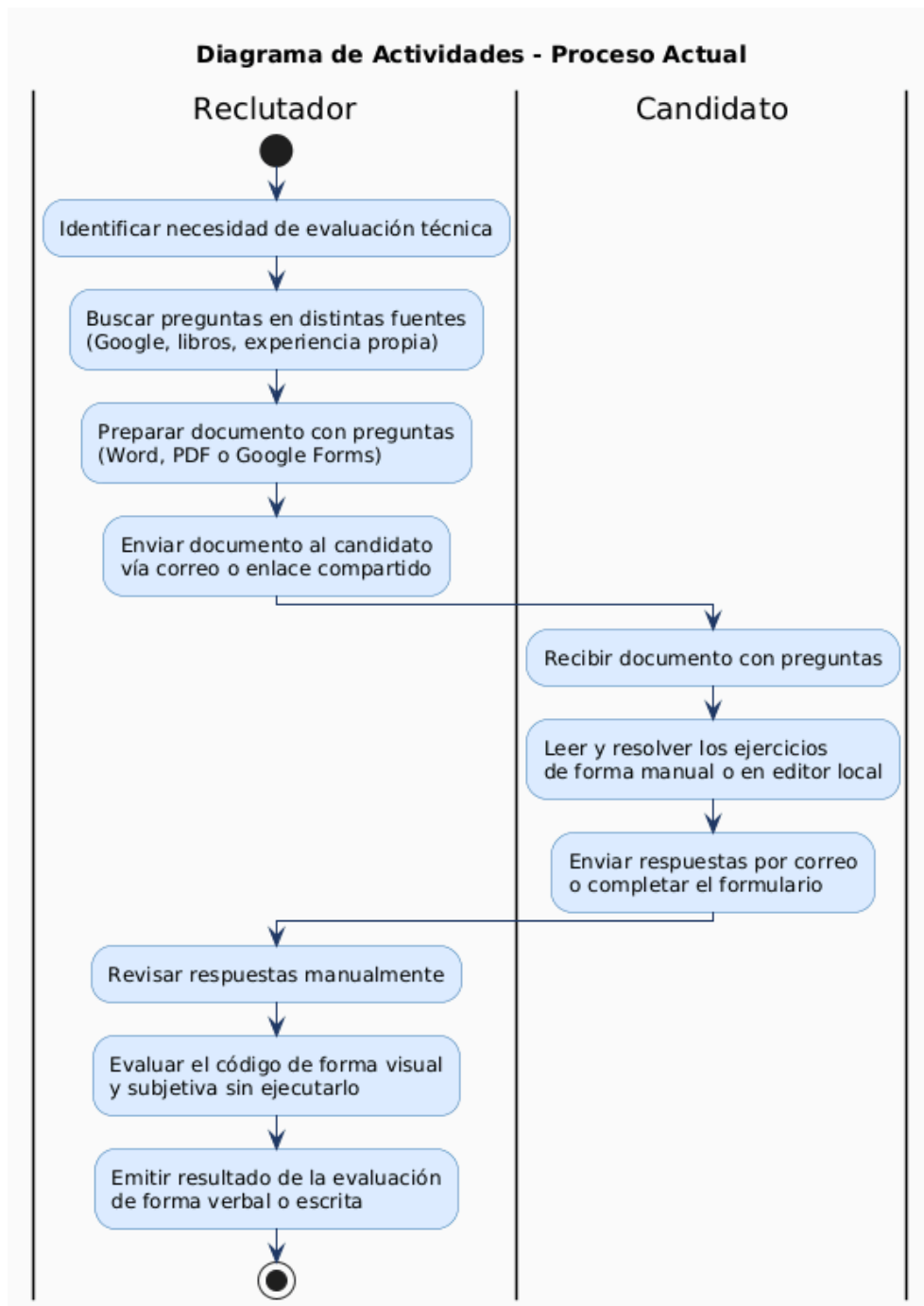
- **Reclutador técnico** de una empresa de desarrollo de software local, quien señaló que el mayor problema es la falta de estandarización en las pruebas: cada entrevistador usa sus propias preguntas sin un repositorio común, y la evaluación del código se hace de forma visual y subjetiva.
- **Desarrollador senior**, quien indicó que como candidato ha enfrentado entornos de evaluación poco realistas (documentos Word, Google Forms) que no reflejan cómo se programa en el trabajo real.
- **Docente universitario de programación**, quien expresó la necesidad de una herramienta que permita asignar ejercicios algorítmicos y verificar automáticamente si la solución es correcta, sin revisar manualmente cada entrega.

Observaciones Se observó el proceso actual de una entrevista técnica en una empresa del sector TI de Tacna, identificando los siguientes problemas:

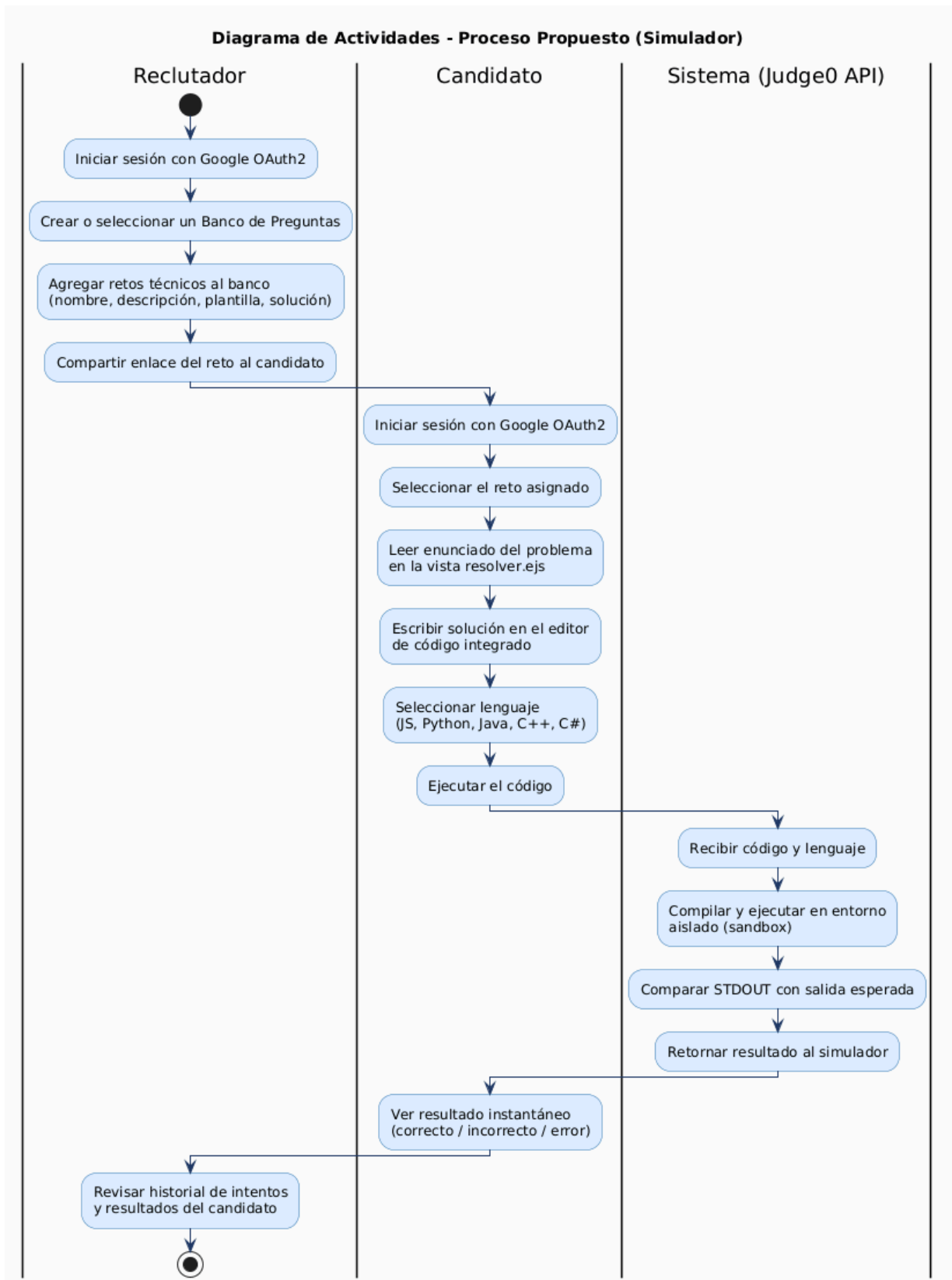
- El reclutador dedica entre 30 y 45 minutos solo en preparar el material de evaluación por candidato.
- No existe registro histórico de qué preguntas se hicieron ni qué resultados obtuvo cada candidato.
- La evaluación del código depende del criterio personal del entrevistador, lo que genera inconsistencias.

III. ANÁLISIS DE PROCESOS

a) Diagrama del Proceso Actual – Diagrama de Actividades



b) Diagrama del Proceso Propuesto – Diagrama de Actividades Inicial



IV. ESPECIFICACIÓN DE REQUERIMIENTOS DE SOFTWARE

a) Cuadro de Requerimientos Funcionales Inicial

ID	Categoría	Descripción
RNF-01	Seguridad	La autenticación debe realizarse exclusivamente mediante tokens OAuth2 verificados con google-auth-library
RNF-02	Seguridad	Las credenciales sensibles (BD, puertos, claves) deben almacenarse en el archivo .env, nunca en el código fuente
RNF-03	Rendimiento	La ejecución del código vía Judge0 debe retornar resultado en menos de 10 segundos
RNF-04	Disponibilidad	El sistema debe estar disponible al menos el 95% del tiempo en horario de uso académico/laboral
RNF-05	Usabilidad	La interfaz debe seguir el tema Dark/Glassmorphism, intuitiva para usuarios con perfil técnico
RNF-06	Mantenibilidad	El código debe seguir arquitectura MVC clara, con separación entre modelos, vistas y controladores
RNF-07	Compatibilidad	El sistema debe funcionar correctamente en navegadores modernos: Chrome, Firefox y Edge
RNF-08	Escalabilidad	La base de datos debe usar pool de conexiones MySQL para soportar múltiples usuarios simultáneos
RNF-09	Portabilidad	El sistema debe poder desplegarse en cualquier servidor con Node.js instalado

b) Cuadro de Requerimientos No Funcionales

ID	Categoría	Descripción
RNF-01	Seguridad	La autenticación debe realizarse exclusivamente mediante tokens OAuth2 verificados con google-auth-library
RNF-02	Seguridad	Las credenciales sensibles (BD, puertos, claves) deben almacenarse en el archivo .env, nunca en el código fuente
RNF-03	Rendimiento	La ejecución del código vía Judge0 debe retornar resultado en menos de 10 segundos
RNF-04	Disponibilidad	El sistema debe estar disponible al menos el 95% del tiempo en horario de uso académico/laboral
RNF-05	Usabilidad	La interfaz debe seguir el tema Dark/Glassmorphism, intuitiva para usuarios con perfil técnico
RNF-06	Mantenibilidad	El código debe seguir arquitectura MVC clara, con separación entre modelos, vistas y controladores
RNF-07	Compatibilidad	El sistema debe funcionar correctamente en navegadores modernos: Chrome, Firefox y Edge
RNF-08	Escalabilidad	La base de datos debe usar pool de conexiones MySQL para soportar múltiples usuarios simultáneos
RNF-09	Portabilidad	El sistema debe poder desplegarse en cualquier servidor con Node.js instalado

c) Cuadro de Requerimientos Funcionales Final

ID	Nombre	Descripción	Cambio respecto al inicial	Prioridad
RF-01	Autenticación con Google	Sin cambios	—	Alta
RF-02	Registro automático de usuario	Sin cambios	—	Alta
RF-03	Crear Banco de Preguntas	Sin cambios	—	Alta
RF-04	Editar Banco de Preguntas	Sin cambios	—	Media
RF-05	Eliminar Banco de Preguntas	Sin cambios	—	Media
RF-06	Listar Bancos de Preguntas	Sin cambios	—	Alta
RF-07	Crear Reto Técnico	Se añade campo de casos de prueba (input/output esperado)	Ampliado	Alta
RF-08	Editar Reto Técnico	Sin cambios	—	Media
RF-09	Eliminar Reto Técnico	Sin cambios	—	Media
RF-10	Listar Retos Técnicos	Se añade filtro por banco de preguntas	Ampliado	Alta
RF-11	Resolver Reto en editor	Sin cambios	—	Alta
RF-12	Seleccionar lenguaje	Sin cambios	—	Alta
RF-13	Ejecutar código via Judge0	Sin cambios	—	Alta

RF-14	Mostrar resultado de ejecución	Sin cambios	—	Alta
RF-15	Historial de intentos	El sistema debe registrar cada intento del candidato con su resultado	Nuevo	Media
RF-16	Cerrar sesión	El usuario debe poder cerrar sesión y eliminar la cookie de sesión activa	Nuevo	Alta

d) Reglas de Negocio

- Solo los usuarios autenticados con Google pueden acceder a la plataforma.
- Un banco de preguntas debe tener al menos un reto para poder ser utilizado en una sesión.
- El código enviado por el usuario es ejecutado en un entorno aislado (sandbox) mediante Judge0.
- La validación de resultados se realiza exclusivamente mediante comparación de salida estándar (STDOUT).
- Los lenguajes de programación soportados son: JavaScript, Python, Java, C++ y C#.

V. FASE DE DESARROLLO

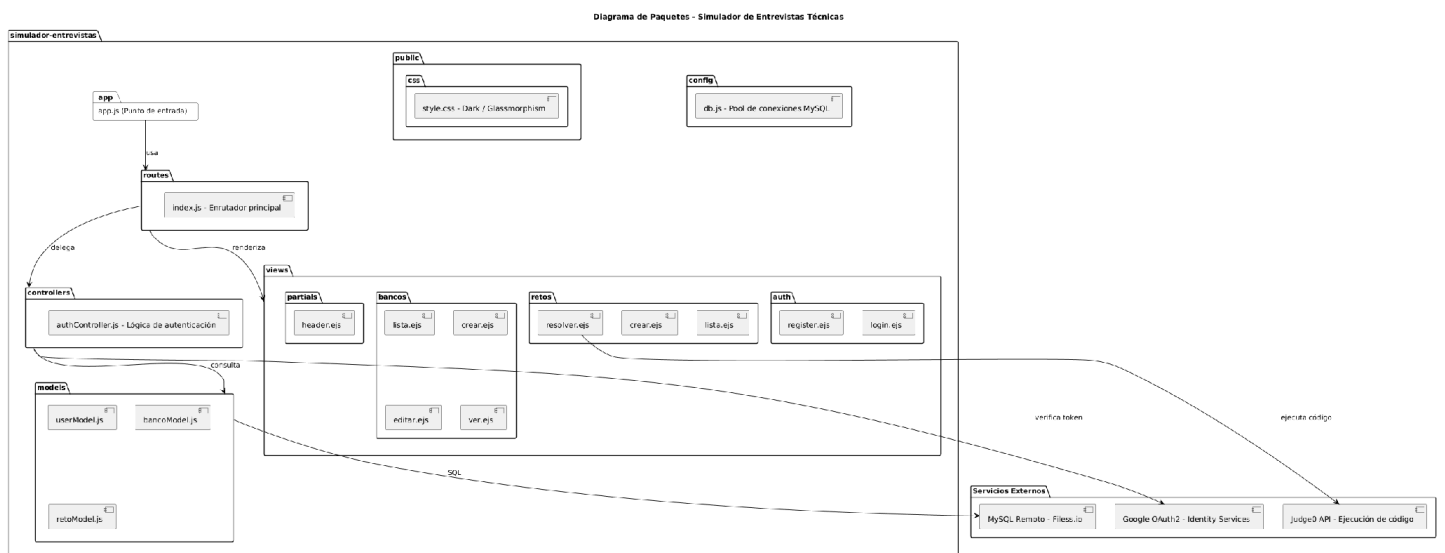
1. Perfiles de Usuario

El sistema contempla dos perfiles principales de usuario:

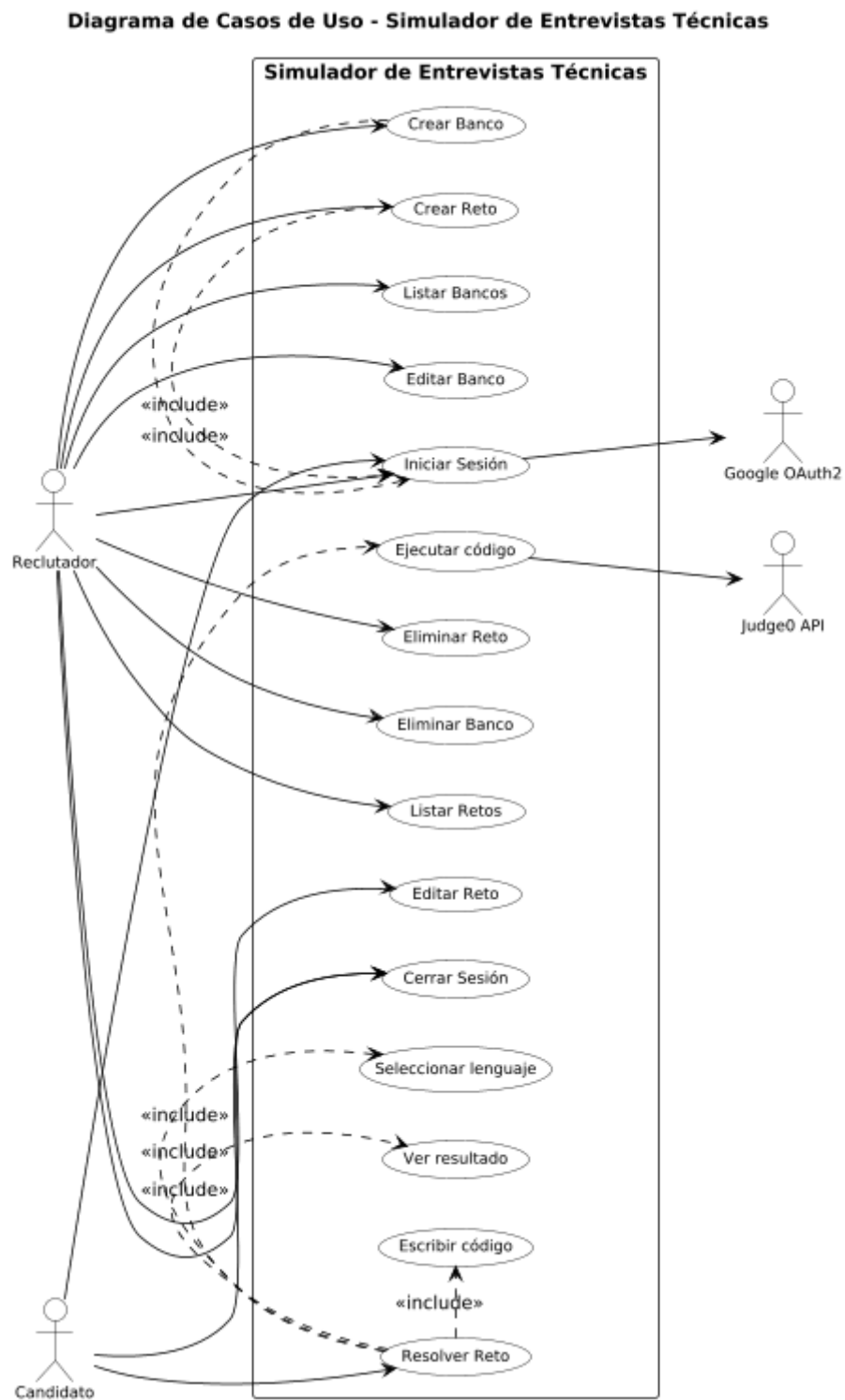
- Reclutador / Administrador: Tiene acceso completo para crear, editar y eliminar bancos de preguntas y retos técnicos.
- Candidato / Desarrollador: Puede visualizar los retos asignados, resolverlos en el editor y ver los resultados de su evaluación.

2. Modelo Conceptual

a) Diagrama de Paquetes



b) Diagrama de Casos de Uso



c) Escenarios de Caso de Uso (Narrativa)

A continuación se describen los escenarios principales de cada caso de uso identificado:

CU-01: Iniciar Sesión con Google

Campo	Descripción
ID	CU-01
Nombre	Iniciar Sesión con Google
Actor Principal	Reclutador / Candidato
Actor Secundario	Google OAuth2
Precondición	El usuario tiene una cuenta de Google válida y activa
Postcondición	El usuario queda autenticado con una sesión activa mediante cookie
Flujo Principal	1. El usuario accede a la pantalla de login (login.ejs)
	2. El usuario hace clic en "Iniciar sesión con Google"
	3. El sistema redirige a Google Identity Services
	4. Google valida las credenciales y retorna un token OAuth2
	5. El sistema verifica el token con google-auth-library
	6. Si el usuario no existe en la BD, se crea automáticamente

	7. El sistema genera una sesión y redirige al dashboard (home.ejs)
Flujo Alternativo	4a. El usuario cancela la autenticación en Google → regresa al login
Flujo de Excepción	5a. El token es inválido → el sistema muestra mensaje de error

CU-02: Cerrar Sesión

Campo	Descripción
ID	CU-02
Nombre	Cerrar Sesión
Actor Principal	Reclutador / Candidato
Precondición	El usuario tiene una sesión activa
Postcondición	La cookie de sesión es eliminada y el usuario es redirigido al login
Flujo Principal	1. El usuario hace clic en "Cerrar sesión" en el header.ejs
	2. El sistema elimina la cookie de sesión del servidor
	3. El sistema redirige al usuario a la pantalla de login
Flujo de Excepción	2a. Si la sesión ya expiró, el sistema redirige directamente al login

CU-03: Listar Bancos de Preguntas

Campo	Descripción
ID	CU-03
Nombre	Listar Bancos de Preguntas
Actor Principal	Reclutador
Precondición	El usuario ha iniciado sesión correctamente
Postcondición	Se muestran todos los bancos registrados en tarjetas (lista.ejs)
Flujo Principal	1. El usuario accede a la sección "Bancos" desde el menú
	2. El sistema consulta bancoModel.js y obtiene todos los bancos
	3. El sistema renderiza la vista lista.ejs con las tarjetas de cada banco
Flujo Alternativo	3a. Si no hay bancos creados → se muestra mensaje "No hay bancos registrados"

CU-04: Crear Banco de Preguntas

Campo	Descripción
ID	CU-04
Nombre	Crear Banco de Preguntas
Actor Principal	Reclutador

Precondición	El usuario ha iniciado sesión correctamente
Postcondición	El nuevo banco queda registrado en la base de datos MySQL
Flujo Principal	1. El usuario accede a "Crear Banco"
	2. El sistema muestra el formulario crear.ejs
	3. El usuario ingresa nombre y descripción del banco
	4. El usuario envía el formulario
	5. El sistema ejecuta la inserción via bancoModel.js
	6. El sistema redirige al listado de bancos
Flujo de Excepción	5a. Si el nombre está vacío → el sistema muestra error de validación

CU-05: Editar Banco de Preguntas

Campo	Descripción
ID	CU-05
Nombre	Editar Banco de Preguntas
Actor Principal	Reclutador
Precondición	Debe existir al menos un banco creado
Postcondición	Los datos actualizados se persisten en la base de datos
Flujo Principal	1. El usuario selecciona "Editar" en un banco existente

	2. El sistema carga los datos actuales en el formulario editar.ejs
	3. El usuario modifica nombre y/o descripción
	4. El usuario guarda los cambios
	5. El sistema ejecuta el UPDATE via bancoModel.js
	6. El sistema redirige al listado de bancos
Flujo de Excepción	5a. Si el nombre queda vacío → el sistema muestra error de validación

CU-06: Eliminar Banco de Preguntas

Campo	Descripción
ID	CU-06
Nombre	Eliminar Banco de Preguntas
Actor Principal	Reclutador
Precondición	Debe existir al menos un banco creado
Postcondición	El banco y sus retos asociados son eliminados de la base de datos
Flujo Principal	1. El usuario hace clic en "Eliminar" en un banco existente
	2. El sistema solicita confirmación al usuario
	3. El usuario confirma la eliminación
	4. El sistema ejecuta el DELETE via bancoModel.js

	5. El sistema redirige al listado actualizado
Flujo Alternativo	3a. El usuario cancela → no se realiza ningún cambio

CU-07: Listar Retos Técnicos

Campo	Descripción
ID	CU-07
Nombre	Listar Retos Técnicos
Actor Principal	Reclutador / Candidato
Precondición	El usuario ha iniciado sesión correctamente
Postcondición	Se muestran todos los retos disponibles en lista.ejs
Flujo Principal	1. El usuario accede a la sección "Retos" desde el menú
	2. El sistema consulta retoModel.js y obtiene todos los retos
	3. El sistema renderiza la vista con nombre, descripción y banco asociado
Flujo Alternativo	3a. Si no hay retos → se muestra mensaje "No hay retos registrados"

CU-08: Crear Reto Técnico

Campo	Descripción
ID	CU-08
Nombre	Crear Reto Técnico
Actor Principal	Reclutador
Precondición	Debe existir al menos un banco de preguntas creado
Postcondición	El reto queda registrado y disponible para ser resuelto
Flujo Principal	1. El usuario accede a "Crear Reto"
	2. El sistema muestra el formulario crear.ejs
	3. El usuario ingresa nombre, descripción, plantilla de código y solución de referencia
	4. El usuario selecciona el banco al que pertenece el reto
	5. El usuario envía el formulario
	6. El sistema registra el reto via retoModel.js
	7. El sistema redirige al listado de retos
Flujo de Excepción	6a. Si no se selecciona banco → el sistema muestra error de validación

CU-09: Editar Reto Técnico

Campo	Descripción
ID	CU-09
Nombre	Editar Reto Técnico
Actor Principal	Reclutador
Precondición	Debe existir al menos un reto creado
Postcondición	Los datos actualizados del reto se persisten en la base de datos
Flujo Principal	1. El usuario selecciona "Editar" en un reto existente
	2. El sistema carga los datos actuales en el formulario editar.ejs
	3. El usuario modifica los campos necesarios
	4. El usuario guarda los cambios
	5. El sistema ejecuta el UPDATE via retoModel.js
	6. El sistema redirige al listado de retos

CU-10: Eliminar Reto Técnico

Campo	Descripción
ID	CU-10
Nombre	Eliminar Reto Técnico
Actor Principal	Reclutador
Precondición	Debe existir al menos un reto creado
Postcondición	El reto es eliminado permanentemente de la base de datos
Flujo Principal	1. El usuario hace clic en "Eliminar" en un reto existente
	2. El sistema solicita confirmación
	3. El usuario confirma la eliminación
	4. El sistema ejecuta el DELETE via retoModel.js
	5. El sistema redirige al listado actualizado
Flujo Alternativo	3a. El usuario cancela → no se realiza ningún cambio

CU-11: Resolver Reto Técnico

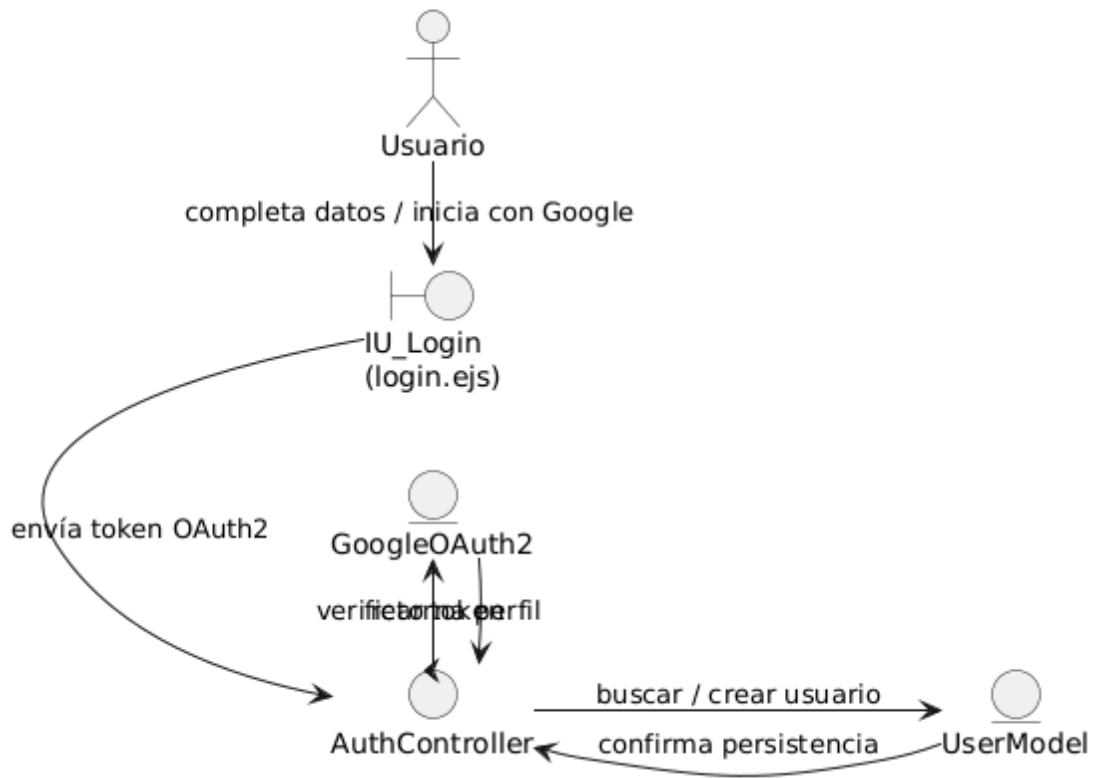
Campo	Descripción
ID	CU-11
Nombre	Resolver Reto Técnico
Actor Principal	Candidato
Actor Secundario	Judge0 API
Precondición	El usuario ha iniciado sesión y existe al menos un reto disponible
Postcondición	El candidato recibe retroalimentación inmediata sobre su solución
Flujo Principal	1. El candidato selecciona un reto desde la lista
	2. El sistema carga la vista resolver.ejs con el enunciado y editor de código
	3. El candidato lee el enunciado del problema
	4. El candidato escribe su solución en el editor integrado
	5. El candidato selecciona el lenguaje de programación
	6. El candidato presiona "Ejecutar"
	7. El sistema envía el código y lenguaje a la API de Judge0

	8. Judge0 compila y ejecuta el código en un entorno aislado
	9. Judge0 compara el STDOUT con la salida esperada y retorna el resultado
	10. El sistema muestra al candidato si su solución es correcta o incorrecta
Flujo Alternativo	8a. El código tiene errores de compilación → Judge0 retorna el mensaje de error y se muestra al candidato
Flujo de Excepción	7a. Judge0 no responde → el sistema muestra mensaje "Error al conectar con el servidor de ejecución"

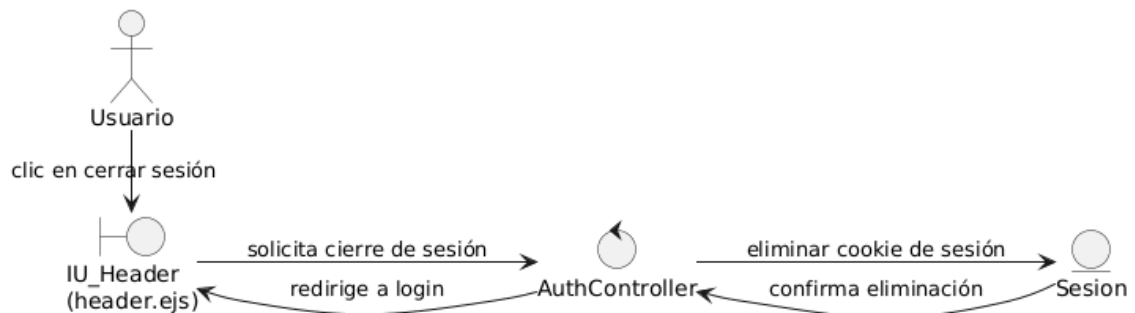
3. Modelo Lógico

a) Análisis de Objetos

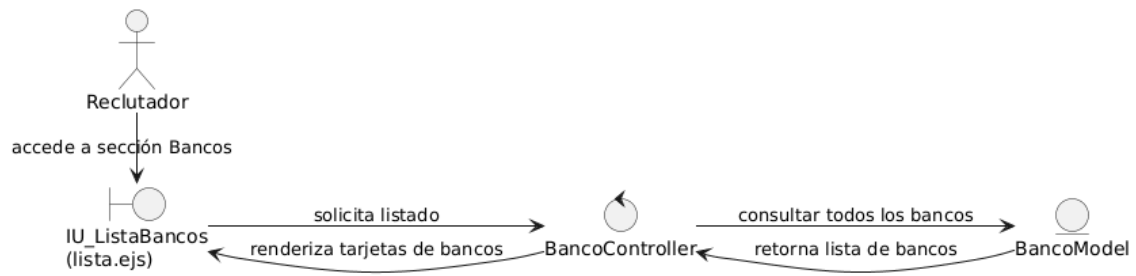
Análisis de Objetos - CU-01: Iniciar Sesión con Google



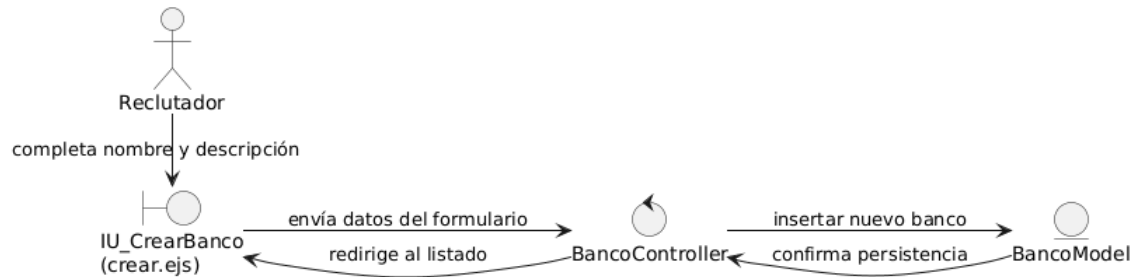
Análisis de Objetos - CU-02: Cerrar Sesión



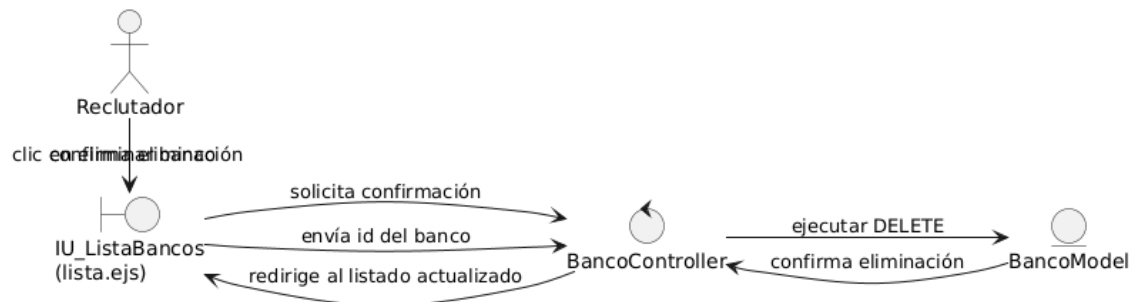
Análisis de Objetos - CU-03: Listar Bancos de Preguntas



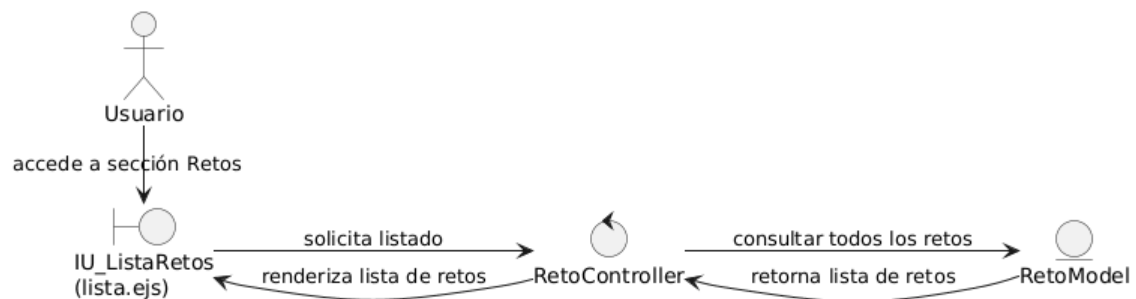
Análisis de Objetos - CU-04: Crear Banco de Preguntas



Análisis de Objetos - CU-06: Eliminar Banco de Preguntas

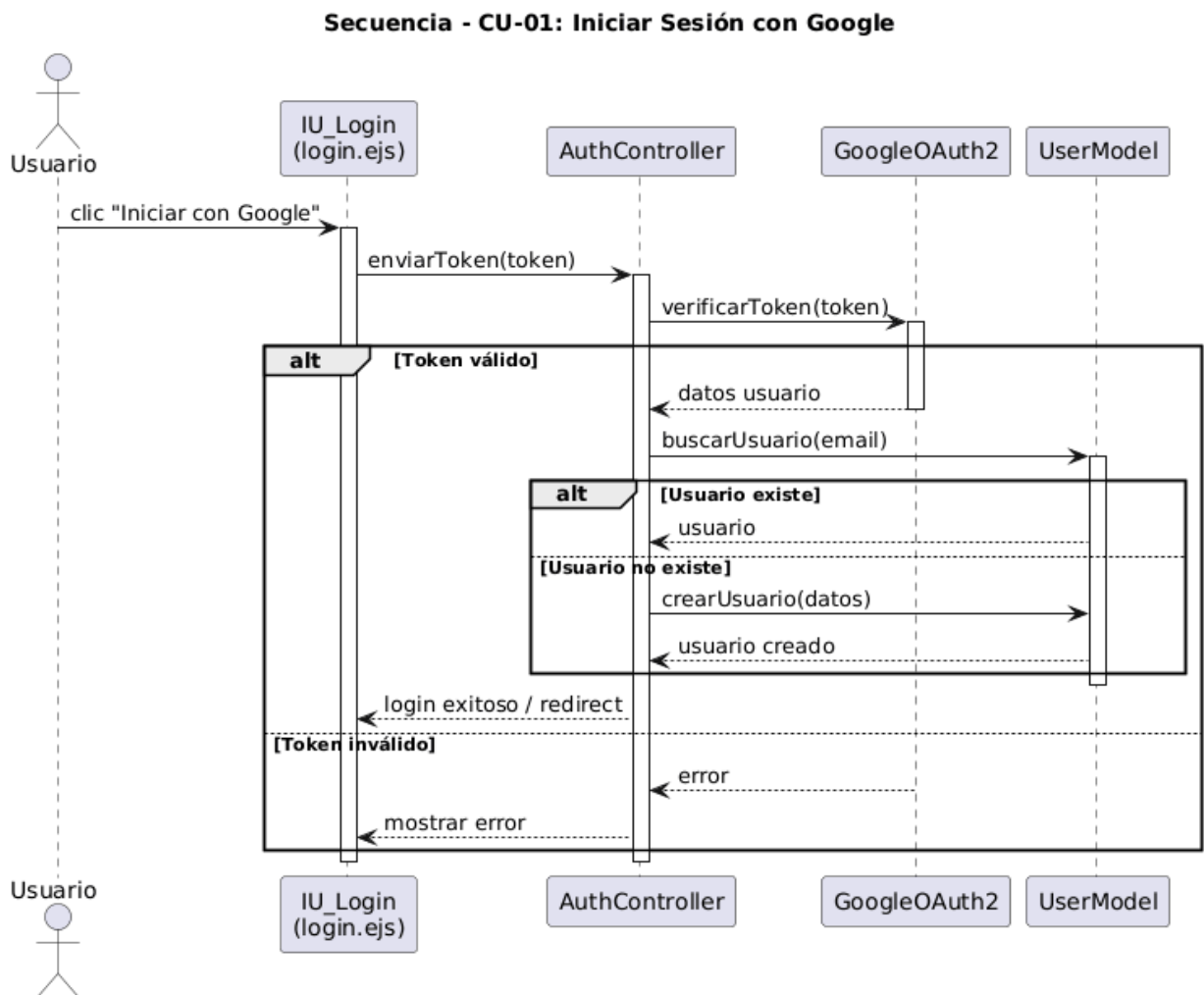


Análisis de Objetos - CU-07: Listar Retos Técnicos



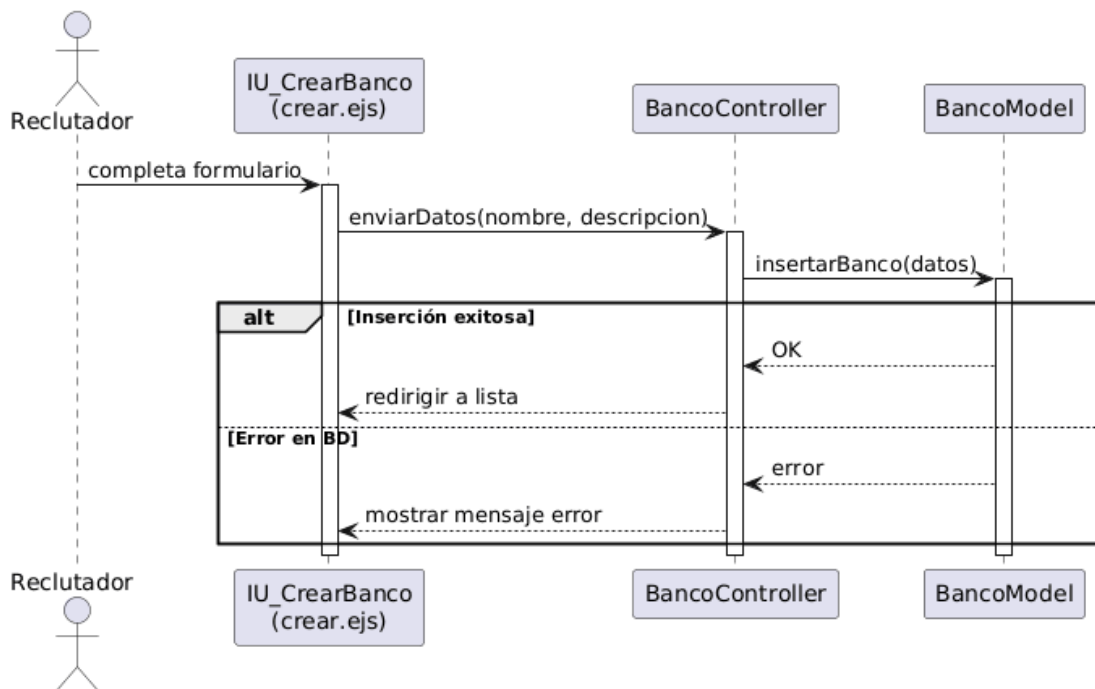
c) Diagrama de Secuencia

CU-01: Iniciar Sesión con Google



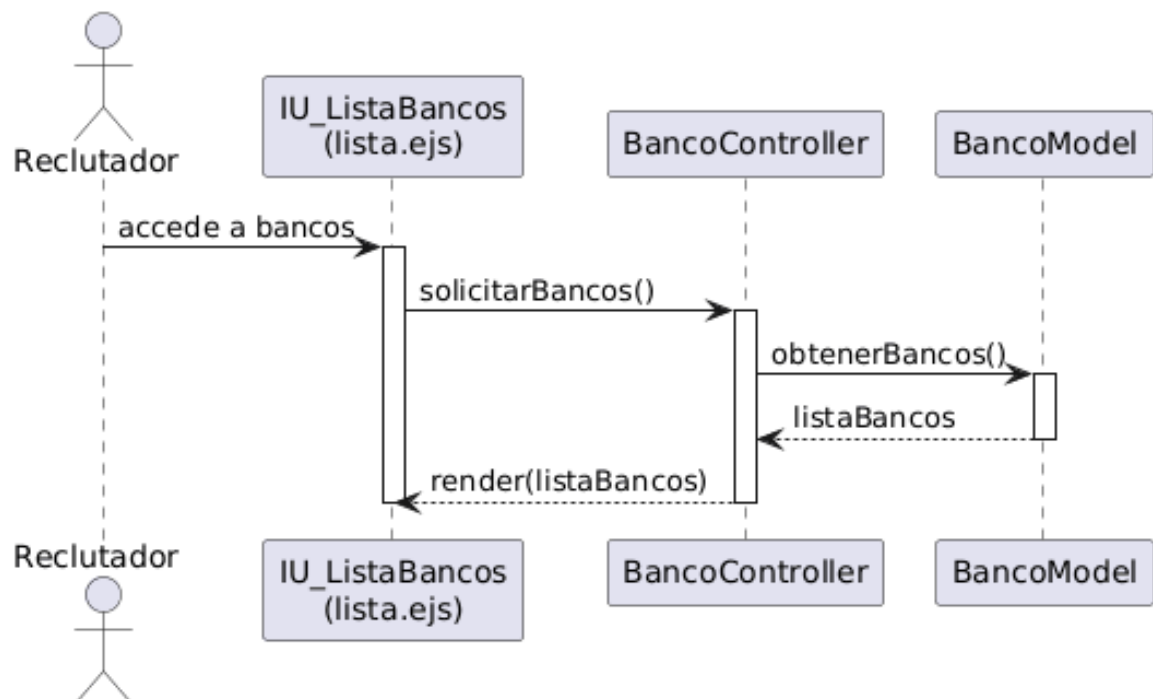
CU-04: Crear Banco de Preguntas

Secuencia - CU-04: Crear Banco de Preguntas



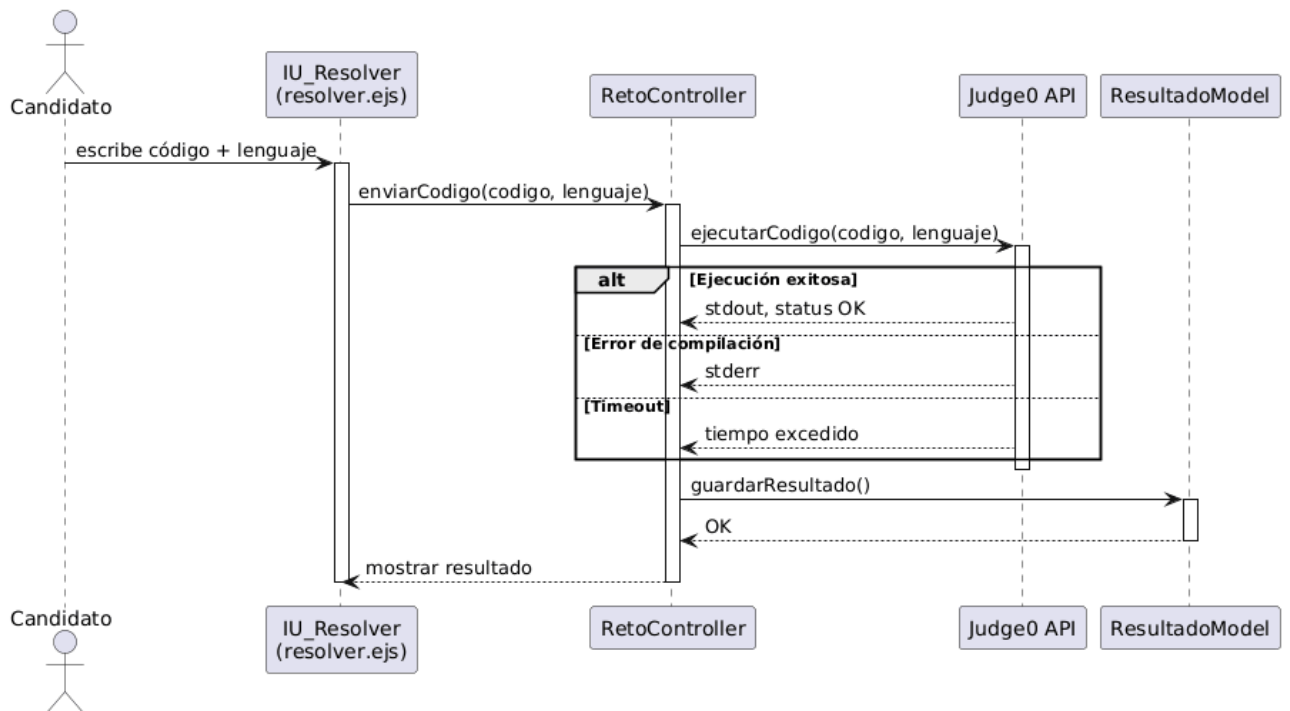
CU-03: Listar Bancos

Secuencia - CU-03: Listar Bancos



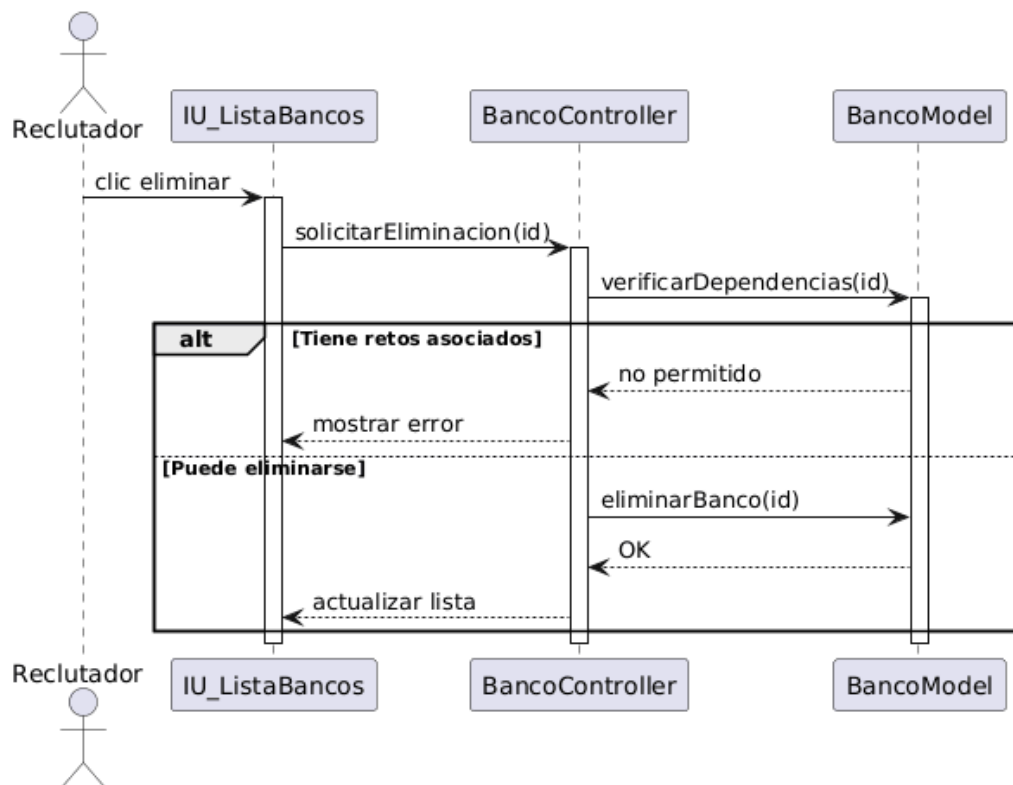
CU-11: Resolver Reto

Secuencia - CU-11: Resolver Reto Técnico



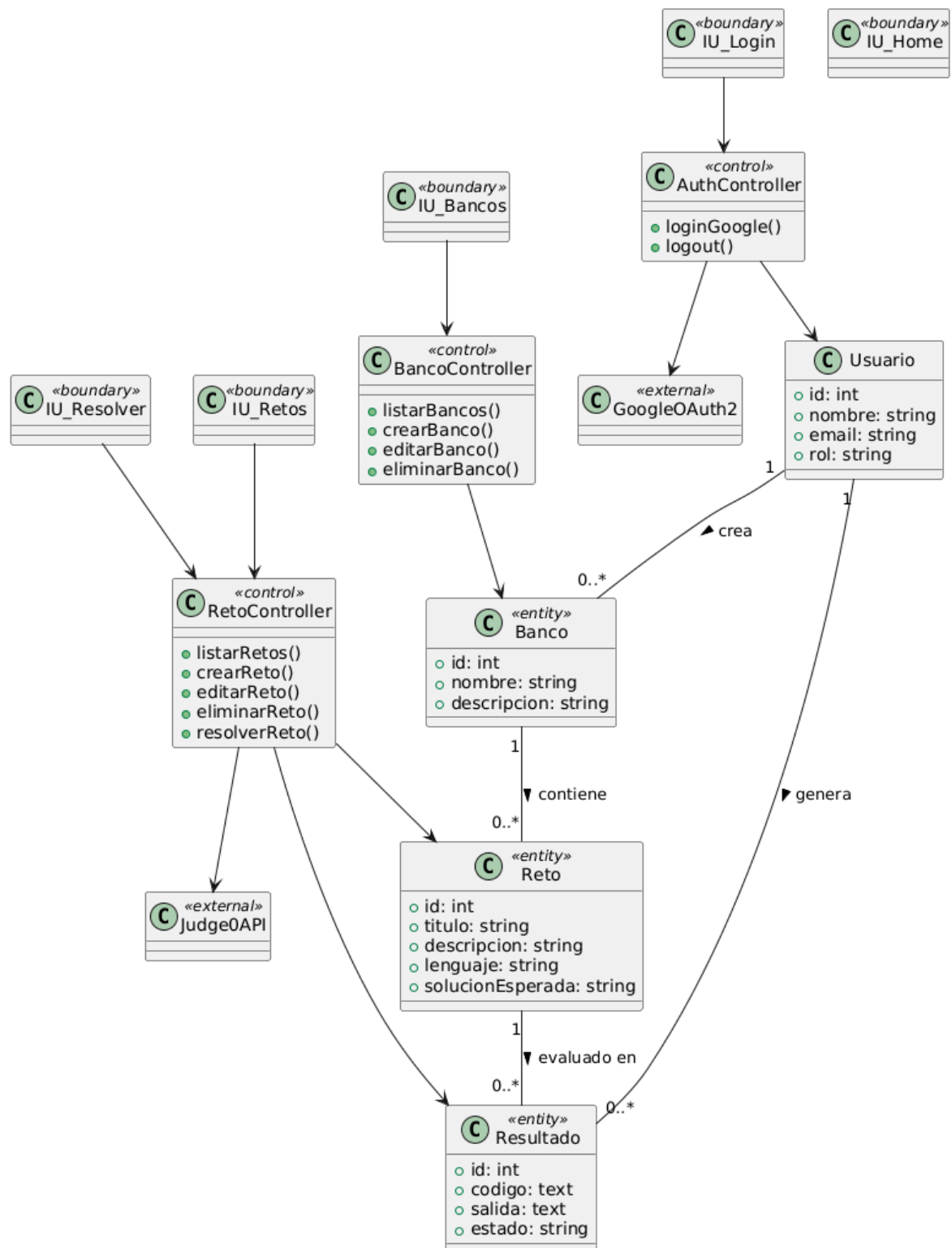
CU-06: Eliminar Banco

Secuencia - CU-06: Eliminar Banco



d) Diagrama de Clases

Diagrama de Clases de Análisis - Simulador de Entrevistas Técnicas



CONCLUSIONES

- El Simulador de Entrevistas Técnicas resuelve una necesidad real del mercado tecnológico, ofreciendo una plataforma centralizada y automatizada para la evaluación de candidatos.
- La integración con la API de Judge0 permite la ejecución de código en tiempo real en múltiples lenguajes, garantizando resultados objetivos e inmediatos.
- La arquitectura MVC con Node.js y Express facilita la escalabilidad y mantenimiento del sistema a largo plazo.
- El uso de autenticación OAuth2 con Google garantiza la seguridad del acceso sin necesidad de gestionar contraseñas propias.

RECOMENDACIONES

- Implementar roles diferenciados (administrador, reclutador, candidato) con permisos específicos para mayor control del sistema.
- Agregar métricas de rendimiento por candidato: tiempo de resolución, intentos y porcentaje de casos superados.
- Migrar la base de datos a un proveedor más robusto (AWS RDS, PlanetScale) para entornos de producción.
- Considerar la incorporación de un sistema de historiales de sesiones para revisión posterior de los resultados.

BIBLIOGRAFÍA

- Barbacci, M., Ellison, R., Lattanze, A., Stafford, J., Weinstock, C., & Wood, W. (2003). *Quality Attribute Workshops (QAWs), Third Edition*. Software Engineering Institute, Carnegie Mellon University.
- Kruchten, P. (1995). *The 4+1 View Model of Architecture*. IEEE Software, 12(6), 42–50.
- Pressman, R. S., & Maxim, B. R. (2014). *Ingeniería de Software: Un enfoque práctico* (8.ª ed.). McGraw-Hill.
- Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.
- Wojcik, R., Bachmann, F., Bass, L., Clements, P., Merson, P., Nord, R., & Wood, B. (2013). *Attribute-Driven Design (ADD), Version 2.0*. Software Engineering Institute.
- Google Developers. (2023). *Google Identity Services Documentation*. Recuperado de <https://developers.google.com/identity>
- Judge0. (2024). *Judge0 API Documentation*. Recuperado de <https://judge0.com/>
- Node.js Foundation. (2024). *Node.js Documentation*. Recuperado de <https://nodejs.org/>
- Oracle. (2024). *MySQL Documentation*. Recuperado de <https://dev.mysql.com/doc/>
- Mozilla Developer Network. (2024). *Web Docs (HTML, CSS, JavaScript)*. Recuperado de <https://developer.mozilla.org/>

WEBGRAFÍA

- Judge0 API Documentation. Recuperado de: <https://judge0.com>
- Google Identity Services – OAuth 2.0. Recuperado de: <https://developers.google.com/identity>
- Express.js Documentation. Recuperado de: <https://expressjs.com>
- MySQL2 Node.js Driver. Recuperado de: <https://github.com/sidorares/node-mysql2>